

SuperGuard: Characterizing and Mitigating Superchip Interference for LLM Serving and Training

Abstract

Superchips promise high-performance AI execution by eliminating traditional host-device bottlenecks. However, we observe a counterintuitive phenomenon: GPU execution frequently stalls under CPU-GPU co-execution even when GPU compute and interconnect bandwidth are not saturated. Through a systemic characterization of modern AI workloads spanning LLM inference and training, we find that CPU-side contention can throttle GPU execution by slowing the host-side kernel dispatch path, leading to GPU idle gaps and significant performance loss. This effect, which we term *Superchip interference*, reveals an overlooked dependency between host-side resource contention and device utilization in tightly coupled hardware. To mitigate this problem, we design SuperGuard, a lightweight runtime that protects the dispatch path through dynamic resource control. SuperGuard monitors dispatcher efficiency and adjusts CPU-side resource allocation to maintain a stable kernel dispatch rate under contention. Across diverse workloads, including RAG-based inference, agentic pipelines, and training with CPU offloading, SuperGuard improves performance by up to 1.8× and recovers up to 95% of the interference-free upper bound, while incurring negligible overhead.

1 Introduction

Efficient execution of modern LLM workloads increasingly depends on CPU-GPU co-execution. In inference, pipelines such as RAG and agentic systems combine GPU computation with CPU-side tasks including data retrieval and external tool execution [1, 8, 10, 25, 29, 30, 35, 67, 70, 71]. In training, techniques such as CPU offloading move model states to host to improve scalability and memory efficiency [21, 33, 46, 49]. These systems demonstrate the growing importance of CPU-GPU co-execution in modern AI workloads.

Tightly coupled CPU-GPU hardware, such as NVIDIA GH200/GB300 Superchips [38, 40, 41], are designed to improve the efficiency of such workloads by integrating CPU and GPU dies within the same package through low-latency, high-bandwidth interconnects such as NVLink-C2C (with up to 900 GB/s bandwidth) [64]. By reducing traditional PCIe-based host-device communication overhead, these hardware are expected to improve CPU-GPU co-execution efficiency.

Despite this promise, we observe that Superchips can fail to deliver the expected efficiency when CPU-side processes and GPU kernels execute concurrently. Across both inference and training workloads on the GH200 Superchip, GPU throughput can drop significantly, even when GPU compute and C2C bandwidth remain not saturated. This behavior

is counterintuitive. Why does this happen on hardware designed to eliminate CPU-GPU communication bottlenecks?

We perform the *first* systematic characterization and root-cause analysis of performance degradation in CPU-GPU co-execution on Superchips in both LLM serving and training (§ 3). We find that the slowdown does not stem from data movement over NVLink-C2C, but from contention in the CPU memory hierarchy. This contention reduces the kernel dispatch rate, throttling GPU execution and leaving the GPU underutilized. We refer to this effect as *Superchip interference*.

This form of interference has been largely overlooked because it is masked in conventional systems. Prior designs typically assume that GPU execution dominates and that CPU-side overheads are effectively hidden. However, as GPUs become faster and CPU-GPU co-execution becomes more intensive, CPU-side contention is increasingly exposed and begins to limit GPU progress. This effect becomes particularly visible on tightly coupled platforms such as Superchips, where fast C2C links enable closer interaction with host.

This effect is increasingly relevant in modern systems, but mitigating it is challenging. First, the source of interference is not directly observable: CPU-side contention arises in shared resources, but its impact manifests itself indirectly through reduced kernel dispatch rate and GPU idle time. Second, modern AI workloads are highly dynamic, with time-varying CPU activity and heterogeneous co-located tasks. Finally, any mitigation must be both accurate and lightweight, as heavy-handed isolation can degrade overall system efficiency.

To address the challenge, we design SuperGuard, a lightweight runtime system that mitigates Superchip interference by protecting the CPU-side dispatch path and stabilizing GPU execution under CPU-GPU co-execution (§ 4). The key idea is to monitor and dynamically control CPU-side resource contention to maintain a stable dispatch rate within a safe operating region. Across LLM workloads, including throughput-oriented training and latency-critical serving scenarios, SuperGuard effectively protects GPU execution and recovers performance, approaching the interference-free upper bound. This paper makes the following contributions:

- **Superchip interference: a new performance phenomenon.** We identify a counterintuitive behavior in Superchips and modern AI workloads, where GPU stalls under CPU-GPU co-execution even when GPU compute and interconnect bandwidth are not saturated, and term this effect *Superchip interference*.
- **First systematic characterization of the root cause.** We present, to the best of our knowledge, the first systematic characterization and root-cause analysis of

Superchip interference under CPU-GPU co-execution, showing that CPU-side memory contention reduces kernel dispatch rate and directly limits GPU progress.

- **SuperGuard: a runtime for mitigating dispatch-path interference.** We design SuperGuard, a lightweight runtime that dynamically controls CPU-side resource usage to protect the dispatch path and maintain a stable dispatch rate under CPU-GPU co-execution.

We implement SuperGuard and evaluate it across both LLM serving and training workloads using vLLM and DeepSpeed over a range of dense and MoE models. SuperGuard improves end-to-end performance by up to 80%, while achieving up to 95% of the interference-free upper bound. Moreover, SuperGuard consistently reduces both average and tail latency in serving workloads, including RAG and agentic SQL pipelines.

2 GPU Stalls Without Bottlenecks

We observe a counterintuitive slowdown under CPU-GPU co-execution on Superchips: GPU performance can degrade significantly when CPU-side processes and GPU kernels execute concurrently, even in the absence of conventional bottlenecks. We demonstrate this effect using both a controlled microbenchmark that isolates CPU-GPU interaction (§2.1) and real-world LLM serving and training workloads (§2.2), and then analyze its root cause.

2.1 A Counterintuitive Observation

Superchips are designed to eliminate traditional PCIe-based CPU-GPU bottlenecks through high-bandwidth, cache-coherent interconnects (e.g., NVLink-C2C). As a result, one would expect GPU performance to be primarily limited by GPU compute or GPU memory bandwidth. We construct a controlled microbenchmark that continuously issues GPU GEMM kernels on preallocated device tensors, while running a concurrent CPU workload that performs large-scale array computation to stress memory and compute resources. The GPU workload mimics common LLM operators (e.g., MLP and attention), while CPU workload intensity is varied by scaling the number of active cores. Intuitively, since the GPU workload remains unchanged across iterations, increasing CPU load should have little impact on GPU throughput. In particular, with no additional GPU computation or data movement, one would expect GPU performance to remain stable. *Why should CPU execution affect GPU throughput at all?*

However, we observe a very different behavior. As shown in Fig. 1(a), increasing CPU workload intensity leads to a 32% drop in GPU throughput (from 746 TFLOPS to 505 TFLOPS), despite no saturation in GPU compute or interconnect bandwidth. A breakdown in Fig. 1(b) further explains this behavior. While kernel execution time remains largely unchanged (blue bars), the host-side kernel launch delay (red bars) increases significantly as CPU load grows. As a result, the GPU spends more time idle between kernel executions (orange bars)

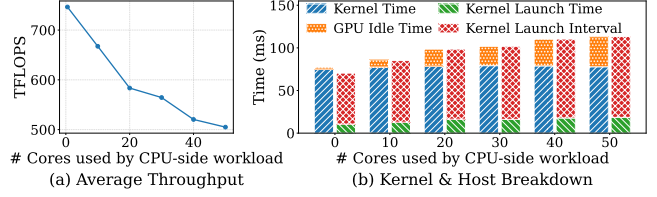


Figure 1. GPU slowdown under concurrent CPU execution on a Superchip. (a) GPU throughput drops by up to 32% as CPU workload intensity increases, despite no change in GPU computation or interconnect usage. (b) Breakdown of GPU execution showing that slowdown is driven by increased host-side launch delay, leading to growing GPU idle time.

bars), indicating that the slowdown is caused by delayed kernel dispatch rather than slower GPU computation. These observations suggest that GPU performance can become limited by host-side execution – a phenomenon that is not observed on conventional GPUs, where GPU throughput remains unaffected under similar conditions (§ 6.4.7).

2.2 Impact on LLM Serving and Training Workloads

We next examine how this phenomenon affects real-world AI workloads, including both LLM serving and training. Representative overlap patterns are illustrated in Appendix A.

2.2.1 RAG-based LLM Inference. Retrieval-Augmented Generation (RAG) pipelines split inference into CPU-side retrieval and GPU-side generation [22, 31]. Modern systems execute these stages asynchronously, allowing retrieval for one request to overlap with generation for another [5, 7, 19, 27, 28, 36, 48, 56, 57, 70]. Under ideal conditions, the two stages should scale independently.

To isolate the effect of CPU execution, we compare a standard RAG pipeline (*Normal*) with a simulated baseline (*Simulated Retrieval*) where CPU workers sleep for pre-recorded retrieval durations and use pre-computed results instead of performing actual vector search, preserving timing and query content while eliminating CPU memory contention. We observe that CPU-side retrieval degrades GPU generation performance. As shown in Fig. 2, time-between-tokens (TBT) increases by $1.16\text{--}1.95\times$ ($1.48\times$ on average), indicating reduced GPU processing rate. Although this slowdown is moderate, it has a disproportionate system-level impact. As request load increases, the reduced processing rate lowers system throughput, leading to queuing amplification. Consequently, time-to-first-token (TTFT) increases by up to $62.4\times$ (from 32ms to 2010ms) under high load. This leads to an important insight: CPU-induced GPU slowdown can cascade into severe end-to-end latency degradation under high load, which is common in production systems. Similar trends hold for other retrieval settings (Appendix B).

2.2.2 LLM Training with CPU Offloading. Modern LLM training systems often offload memory-intensive operations,

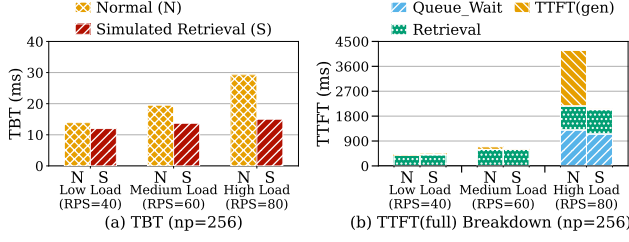


Figure 2. CPU-side retrieval degrades GPU performance in RAG-based inference. (a) Time-between-token (TBT) increases under CPU-side retrieval across different request rates. (b) Breakdown of time-to-first-token (TTFT) showing that modest GPU slowdown leads to severe latency amplification under high load due to queuing effects.

such as optimizer states and updates, to the CPU while retaining compute-intensive operations on the GPU [33, 49]. These systems typically overlap CPU-side computation with GPU execution to hide latency.

To isolate the effect of CPU execution, we compare against a simulated baseline that preserves all GPU-CPU communication while eliminating CPU computation. Specifically, in this baseline, parameters are still transferred between GPU and CPU at the same points in the training pipeline, but instead of computing optimizer updates, the CPU worker sleeps for an equivalent duration. We observe that CPU-side execution significantly degrades GPU performance. As shown in Fig. 3, GPU backward time increases by 19%–87% under CPU optimizer execution across different models and batch sizes. Profiling shows that this slowdown is not caused by increased computation, but by prolonged idle gaps between GPU kernels. Removing CPU-side execution eliminates the slowdown while preserving the same timing and communication pattern, indicating that the degradation is not caused by data transfer overhead. Instead, the degradation arises from CPU–GPU interaction rather than the overlap itself.

2.2.3 Not What You Think. The slowdowns observed above cannot be explained by conventional bottlenecks. First, they are not caused by GPU computation. Across both inference and training, GPU workload remains unchanged, yet execution time increases under concurrent CPU execution. Second, they are not due to interconnect bandwidth limitations. We observe no increase in data transfer volume, and replacing CPU execution with a sleep of equal duration eliminates the slowdown despite preserving communication patterns. Third, they are not caused by kernel inefficiency. The same kernels achieve normal performance when CPU-side execution is removed, and the slowdown persists across workloads and implementations.

3 The Hidden Bottleneck in Tightly Coupled Superchips

The results in § 2 rule out conventional bottlenecks such as GPU computation, interconnect bandwidth, and kernel

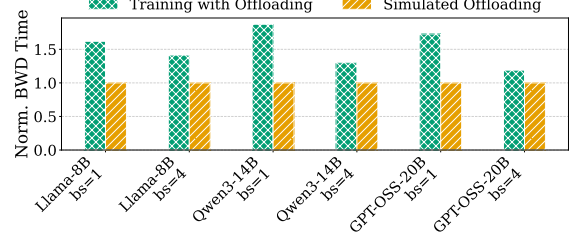


Figure 3. CPU-induced slowdown in GPU backward propagation under offloading training across different models and batch size (bs). GPU backward time increases by 19%–87%, compared to a simulated baseline.

inefficiency. Instead, they point to a previously overlooked dependency between CPU-side execution and GPU progress. In particular, GPU execution depends on the CPU’s ability to sustain a sufficiently high rate of kernel dispatch under tightly coupled CPU–GPU co-execution. We refer to this mechanism as *dispatch-path coupling*.

3.1 The Kernel Dispatch Path

Despite advances in GPU design, execution in modern software stacks remains host-driven. Modern ML frameworks rely on asynchronous execution to overlap host-side management with GPU execution: the CPU prepares and enqueues kernel launches to CUDA streams, while the GPU dequeues and executes them independently [39]. The dispatch process involves preparing kernel metadata (e.g., function pointers, launch configurations, and memory bindings), followed by runtime scheduling and enqueueing. Although these operations are lightweight in computation, they are latency-sensitive and depend heavily on memory efficiency.

However, this mechanism relies on a key assumption: the CPU can supply kernels fast enough to sustain GPU execution. If this assumption is violated, GPU performance suffers. When CPU-side dispatch becomes slower than GPU execution, the stream queue can drain, forcing the GPU to idle waiting for new kernels. As illustrated in Fig. 4, this exposes kernel dispatch latency that would otherwise remain hidden.

Key Insight: Even under asynchronous execution, GPU progress depends on the host’s ability to sustain a sufficiently high kernel dispatch rate.

3.2 From Memory Contention to Dispatch Slowdown

We next connect CPU-side memory pressure to the observed GPU slowdown through the kernel dispatch path. When concurrent CPU workloads generate memory pressure, they increase cache miss rate and DRAM access latency. This slows down dispatch operations and reduces the rate at which kernels can be issued. We observe this effect as a reduction in CPU execution efficiency. As shown in Fig. 5, the IPC (Instructions Per Cycle) of the dispatch process drops by 20.6%–53.3% (39.9% on average), indicating that dispatch becomes increasingly memory-bound.

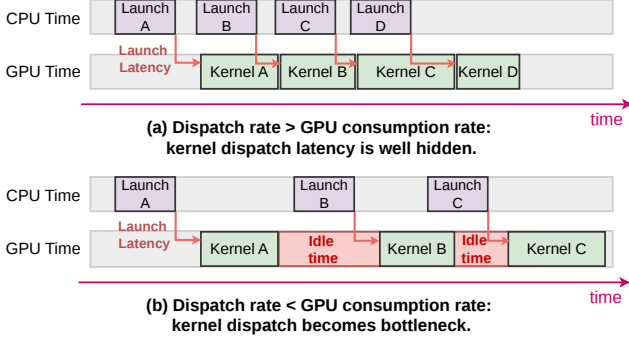


Figure 4. When asynchronous execution breaks: kernel dispatch becomes the bottleneck. (a) Under normal conditions, dispatch latency is hidden by asynchronous execution. GPU remains continuously busy. (b) Under dispatch-path coupling, increased host-side latency reduces dispatch rate, draining the stream queue and introducing GPU idle gaps.

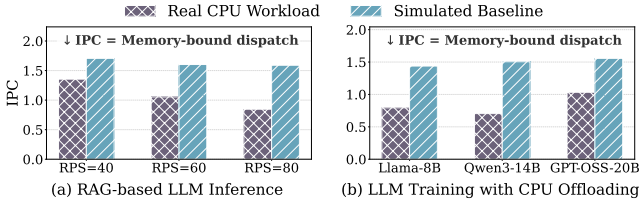


Figure 5. Memory contention reduces CPU dispatch efficiency. The IPC of the CPU-side dispatch process decreases under concurrent CPU workloads in both inference and training. This drop reflects increased memory stalls, reducing the kernel dispatch rate and ultimately limiting GPU progress. RAG uses $nprobe=256$; training uses $bs=4$.

A reduced dispatch rate directly impacts GPU progress. As CPU becomes slower at issuing kernels, CUDA stream queue is no longer continuously supplied. Once the dispatch rate falls below the GPU’s consumption rate, the queue drains, exposing idle gaps between kernel executions. This mechanism explains the primary source of slowdown observed in § 2. In inference, memory-intensive retrieval (e.g., high $nprobe$) increases CPU-side memory pressure, reducing dispatch rate and degrading GPU throughput. In training, optimizer execution introduces sustained memory traffic, similarly slowing dispatch and increasing backward time. Overall, memory contention propagates through the dispatch path, converting host-side latency into GPU idle time. This chain of effects, i.e., memory pressure increasing dispatch latency, reducing dispatch rate, and ultimately introducing GPU stalls, defines *dispatch-path coupling*.

3.3 Evidence: Microarchitectural Characterization

We next examine how memory contention affects the CPU-side dispatch process at the microarchitectural level. The Grace CPU and GPU runtime share a unified last-level cache (LLC) and a common DRAM subsystem. CPU workloads with poor locality can evict dispatcher working sets from

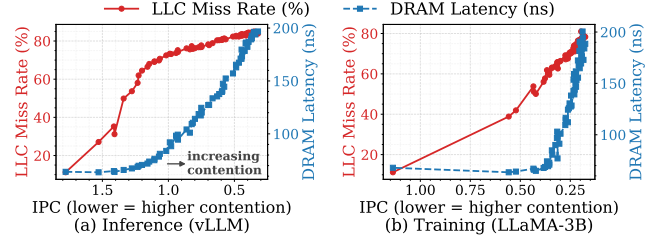


Figure 6. Memory contention drives dispatch to become memory-bound. As memory contention increases, LLC miss rate rises sharply and DRAM access latency increases (up to $\sim 3\times$), while host-side dispatch IPC drops significantly. This demonstrates that dispatch performance is dominated by memory stalls under contention, limiting dispatch rate.

the LLC, while memory-intensive tasks can saturate DRAM bandwidth and introduce queuing delays at the memory controller. Both effects increase dispatcher stall time and reduce dispatch throughput, directly impacting GPU utilization.

3.3.1 CPU Execution Efficiency (IPC). We first examine the execution efficiency of the host-side dispatch process. The results in § 3.2 show that the IPC of the dispatch process drops significantly under concurrent CPU execution. To further isolate the effect of memory contention, we design a controlled stress experiment that we introduce additional DRAM bandwidth pressure via a concurrent CPU workload while running vLLM inference and LLM training. As shown in Fig. 6, under this setting, IPC drops more dramatically, by up to 82% (from 1.78 to 0.32) for inference and 83% (from 1.14 to 0.19) for training, demonstrating that the dispatch process is highly sensitive to memory system interference. This indicates that the dispatch process becomes increasingly memory-bound under contention. The reduced IPC reflects increased stall cycles due to memory accesses, which directly slows down kernel dispatch and limits GPU progress.

3.3.2 LLC Contention and Memory Pressure. We next examine how memory contention manifests as increased pressure on the CPU memory hierarchy. As shown in Fig. 6, the LLC miss rate and DRAM access latency of the host-side dispatch process as memory contention increases. As contention grows, the LLC miss rates rises significantly, from approximately 11% to 75-83%, indicating that a large fraction of memory access bypass the last-level cache and are served by DRAM. As a result, DRAM bandwidth utilization raises significantly. We observe similar behavior in real LLM workloads. As shown in Fig. 7, LLC miss rates increase substantially during both RAG-based inference and LLM training. In inference, CPU-side retrieval over large vector databases causes significant cache pressure, increasing LLC miss rate as high as 47%. In training, optimizer execution introduces sustained memory traffic, resulting in LLC miss rates of 21-25%, compared to less than 10% in the simulated baseline that eliminates CPU-side computation. Together,

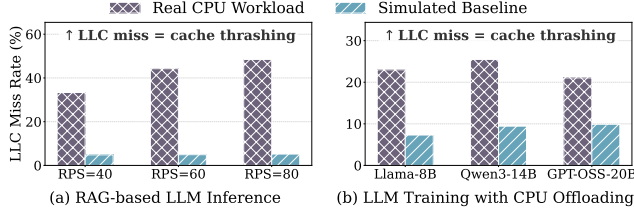


Figure 7. Real LLM workloads induce substantial LLC contention. The host-side dispatch process exhibits significantly higher LLC miss rates during RAG-based inference and LLM training compared to the interference-free simulated baseline, indicating that AI workloads naturally generate the memory pressure that degrades dispatch efficiency.

these results show that both controlled stress and practical workloads generate significant memory pressure, characterized by high LLC miss rates and elevated DRAM bandwidth utilization. However, increased memory pressure alone does not fully explain the sharp slowdown of the dispatch process. In particular, it remains unclear why memory access latency increases so dramatically under contention.

3.3.3 DRAM Queuing Effects and Latency Inflation.

We find that the sharp increase in memory access latency is primarily driven by queuing effects in the Grace CPU DRAM subsystem. As DRAM bandwidth utilization increases, memory requests accumulate at the memory controller, leading to longer waiting times before service. This queuing effect introduces a non-linear relationship between bandwidth utilization and access latency, a phenomenon also previously observed in CPU-only systems [14, 23, 37, 50]. Fig. 8 illustrates this relationship between bandwidth utilization and DRAM latency on the GH200 Superchip. Once bandwidth exceeds a moderate threshold (e.g., 100GB/s), DRAM latency increases rapidly, by up to 3.2 $\times$ on average and 4.5 $\times$ at the P90 tail latency, due to queuing delays.

This helps explain the latency increase observed in Fig. 7. Under high memory contention, elevated LLC miss rates lead to more frequent DRAM accesses, increasing bandwidth utilization. As a result, queuing delays significantly inflate the latency of each memory access, amplifying memory stall time. In real workloads, we observe similar high DRAM bandwidth utilization (e.g., 77-175 GB/s for training and up to 300 GB/s for RAG), pushing the memory subsystem to the high-latency regime. Additional breakdowns of bandwidth and latency across configurations are provided in Appendix C.

3.4 Implications for System Design

The above analysis reveals several important implications for system design on tightly coupled Superchips. **Insight-I:** *CPU and GPU execution are not independent.* Even under asynchronous execution, CPU-side memory behavior can directly limit GPU progress through the dispatch path. **Insight-II:** *Overlap is not free.* Memory-intensive CPU workloads can degrade dispatch efficiency and introduce GPU idle time, even

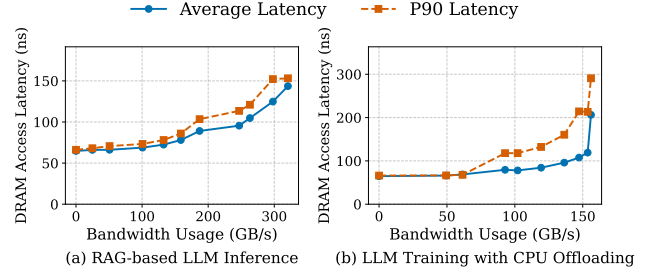


Figure 8. DRAM queuing causes non-linear latency inflation under high bandwidth utilization. As DRAM bandwidth increases, average and P90 access latency rise sharply due to queuing effects in the memory controller. This non-linear latency growth (up to $\sim 3.2\times$ average and $\sim 4.5\times$ tail latency) explains the increase in memory stall time under contention.

when GPU compute and interconnect bandwidth remain underutilized. **Insight-III:** *The dispatch path becomes a critical bottleneck under tightly coupled CPU-GPU co-execution.* Sustaining a stable kernel dispatch rate under memory contention is essential for achieving high GPU utilization. These insights motivate system designs that decouple kernel dispatch from CPU memory interference.

4 SuperGuard Design

The analysis in Section 3 shows that GPU slowdown arises when CPU-side contention reduces the kernel dispatch rate below the GPU’s consumption rate. This suggests that sustaining a sufficiently high dispatch rate is critical for maintaining GPU utilization under CPU-GPU co-execution.

Design objective. SuperGuard aims to maintain a stable kernel dispatch rate sufficient to sustain continuous GPU execution. When the dispatch rate falls below the GPU’s consumption rate, GPU idle gaps emerge. Therefore, SuperGuard seeks to keep the dispatch process within a safe operating region where GPU execution remains uninterrupted.

4.1 Design Overview

SuperGuard formulates CPU-GPU interference as a feedback control problem over the dispatch path. Fig. 9 illustrates the design of SuperGuard. The top panels show the effect of SuperGuard, where kernel dispatch becomes stable, and GPU idle gaps are eliminated compared to the interference case.

As shown in the top portion of the figure, the system separates kernel dispatch (K-Disp) and CPU computation (C-Comp) into distinct resource partitions, while a global allocator (bottom-right) dynamically balances their resource demands based on runtime signals, e.g., dispatcher IPC (§ 4.2) and LLC usage, interpreted through a finite-state policy (e.g., *Supply, Maintain, Demand*). As shown in the bottom panel, SuperGuard operates in two phases: an initial profiling phase that identifies a system-specific safe operating region based on hardware and workload characteristics (§ 4.3), followed by a runtime control phase that enforces this region under

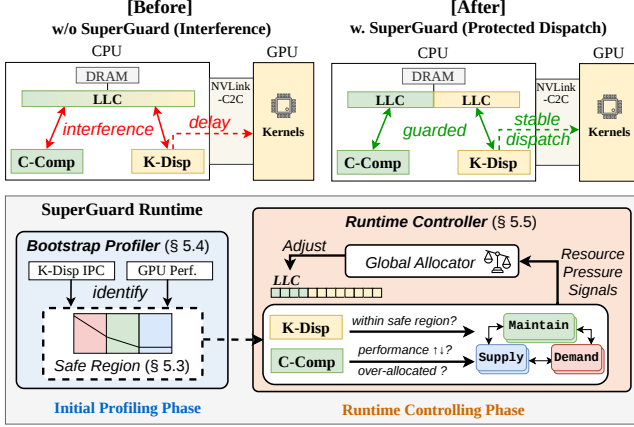


Figure 9. Overview of SuperGuard on a Superchip architecture. CPU and GPU interact through a coherent, tightly coupled memory system, where contention in shared resources (e.g., DRAM and cache hierarchy) can introduce GPU-CPU interference. SuperGuard organizes execution into kernel dispatcher and CPU computation partitions and uses a feedback control loop to dynamically balance resource allocation.

dynamic workloads. Intuitively, this region corresponds to the range of IPC values where the dispatcher is sufficiently provisioned to sustain continuous GPU execution without excessive resource allocation.

The system consists of two main components: (1) a bootstrap profiler that determines the safe operating region (§ 4.4), and (2) a runtime controller that maintains this region by adjusting resource allocation based on dispatcher IPC and partition-level signals (e.g., LLC usage) (§ 4.5). To support this design, SuperGuard implements a lightweight runtime and enforces isolation between kernel dispatch and CPU computation using hardware-supported cache allocation (§ 5). Together, these components form a feedback loop that sustains dispatch-path efficiency under dynamic interference.

4.2 Dispatcher IPC as an Interference Signal

Many LLM applications exhibit dynamic workload patterns, causing application-level latency metrics (e.g., TBT, TTFT) to fluctuate significantly. However, these metrics are influenced by multiple factors, including batching, queuing, and request dynamics, making them less directly aligned with the root cause of Superchip interference. In particular, latency reflects downstream effects such as GPU idle time and queuing delays, rather than the dispatch slowdown itself, and is therefore less suitable for timely control.

For effective control, we require a system-level signal that is both observable and directly tied to the root cause of interference. We find that the execution efficiency of the GPU kernel dispatcher provides such a signal. When the dispatcher operates sufficiently fast, kernel dispatch latency is effectively hidden by GPU execution. However, once the dispatcher performance falls below a critical threshold, the

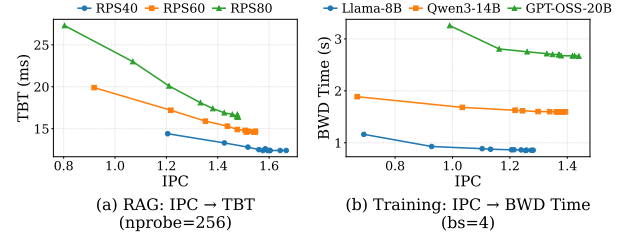


Figure 10. Dispatcher IPC directly reflects the execution efficiency of the dispatch path and exhibits a clear threshold effect on GPU performance.

GPU begins to idle while waiting for new kernels, leading to rapid performance degradation.

This behavior exhibits a clear threshold effect: GPU performance remains stable when dispatcher IPC is above a certain level, but degrades sharply once IPC falls below this threshold. This threshold is primarily determined by the balance between dispatch overhead and GPU execution time, which depends on both system characteristics and workload properties. As a result, the threshold is relatively stable within a given workload regime, but may shift when the dispatch-to-compute balance changes significantly. As shown in Fig. 10, reducing cache allocation gradually decreases dispatcher IPC, while GPU performance remains stable until IPC crosses the threshold, after which performance drops sharply. This behavior confirms that dispatcher IPC is an effective signal for detecting CPU-GPU interference. Additional results across workload configurations are provided in Appendix D.

4.3 Safe Operating Region

Motivated by the threshold effect of dispatcher IPC, SuperGuard defines a safe operating region over IPC, which serves as the control signal for dispatch-path efficiency (§ 4.2). Rather than targeting a single operating point, SuperGuard identifies an IPC interval within which the dispatcher is sufficiently provisioned to sustain continuous GPU execution without unnecessary resource allocation. When IPC falls below a lower bound (ipc_l), the dispatch path becomes under-provisioned, increasing the risk of GPU stalls. Conversely, when IPC exceeds an upper bound (ipc_h), additional resources provide diminishing benefit and can be reclaimed to improve the performance of co-located CPU workloads. This region captures the empirical relationship between dispatcher IPC and application-level performance, where performance degrades rapidly below ipc_l but exhibits diminishing returns beyond ipc_h . Accordingly, SuperGuard formulates interference mitigation as a feedback control problem, where the runtime controller continuously adjusts resource allocation to maintain dispatcher IPC within this safe region.

4.4 Bootstrap Profiling

The bootstrap profiler determines the safe operating region using dispatcher IPC as the interference signal (§ 4.2). The goal is to identify a coarse operating region where dispatch

Algorithm 1: Bootstrap Profiling

```

1 Function Profiling( $W_{\max}, \epsilon_{\text{sat}}, \epsilon_{\text{deg}}$ ):
  // Sweep LLC allocation and collect performance
  samples
2 for  $w \leftarrow W_{\max}$  downto 1 do
3    $S[w] \leftarrow \text{Measure}(\text{dispatcher: } w \text{ LLC ways})$ 
  // Establish baseline from max allocation
4  $s_0 \leftarrow S[W_{\max}]$ 
5  $\text{lat}_{\text{sat}} \leftarrow s_0.\text{lat} \cdot (1 + \epsilon_{\text{sat}})$ 
6  $\text{lat}_{\text{deg}} \leftarrow s_0.\text{lat} \cdot (1 + \epsilon_{\text{deg}})$ 
  // Find safe region boundaries
7  $\text{ipc}_h \leftarrow s_0.\text{ipc}, \text{ipc}_l \leftarrow S[1].\text{ipc}$ 
8 for  $w \leftarrow W_{\max}$  downto 1 do
9   if  $S[w].\text{lat} \leq \text{lat}_{\text{sat}}$  then  $\text{ipc}_h \leftarrow S[w].\text{ipc}$ 
10  if  $S[w].\text{lat} \geq \text{lat}_{\text{deg}}$  then  $\text{ipc}_l \leftarrow$ 
    Interpolate( $S[w-1], S[w]$  at  $\text{lat}_{\text{deg}}$ ) and break
11 return ( $\text{ipc}_l, \text{ipc}_h$ )

```

transitions from insufficient to sufficient to sustain continuous GPU execution. During profiling, measurements are collected under controlled conditions where workload behavior is relatively stable over short intervals. The profiler first provisions the dispatcher with sufficient CPU cores to ensure it is not compute-bound. In practice, this requires only a small number of cores, leaving most CPU resources available for the computation partition. This observation holds across both training and inference workloads (Appendix E).

With sufficient cores provisioned, the profiler constructs the safe operating region (described in § 4.3) by sweeping LLC allocation from maximum to minimum and observing dispatcher IPC along with coarse GPU-side performance indicators (e.g., latency or GPU utilization from `nvidia-smi`) (Algorithm 1). These indicators are used to detect the transition point where GPU execution becomes limited by dispatch.

As discussed in § 4.2, dispatcher IPC exhibits a threshold effect on GPU performance (Fig. 11). Based on this, we define the safe region as $[\text{ipc}_l, \text{ipc}_h]$, where ipc_l corresponds to the point at which GPU execution begins to degrade beyond a threshold of $(1 + \epsilon_{\text{deg}})$ relative to the baseline, and ipc_h corresponds to the point beyond which additional resources yield diminishing returns, defined by a saturation threshold $(1 + \epsilon_{\text{sat}})$. The baseline is established using performance under maximum LLC ways, approximating interference-free execution (lines 4-6 in Algorithm 1).

In practice, the stability of this region depends on workload characteristics. Workloads with more stable execution patterns (e.g., training) exhibit more consistent safe regions, while more dynamic workloads (e.g., RAG-based inference or agentic pipelines), may show higher variability due to dynamic control flow and diverse CPU behavior. Nevertheless, our runtime evaluation (§ 6) shows that the derived region remains sufficiently stable for effective control in practice.

4.5 Runtime Controller

At runtime, SuperGuard employs a lightweight feedback controller to maintain a sufficient GPU dispatch rate while

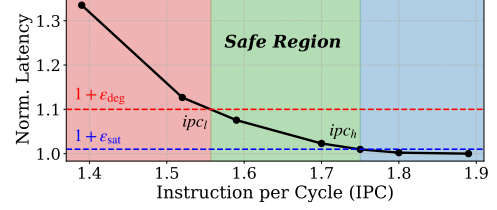


Figure 11. Dispatcher IPC threshold effect on GPU performance. The safe region $[\text{ipc}_l, \text{ipc}_h]$ (green) is used for runtime control. Latency normalized to maximum LLC allocation.

minimizing over-provisioning of shared CPU resources. As shown in Fig. 12, the controller operates as a closed loop with three stages. The controller continuously monitors runtime performance of the kernel dispatcher and CPU computation partitions (❶). These measures are translated into resource pressure signals for each partition via two finite-state machines (FSM), which classify each partition into one of three states: *Demand*, *Maintain*, *Supply* (❷). Based on these states, the controller determines resource allocation decisions (❸).

For the kernel dispatcher partition (left FSM in Fig. 12), the FSM encodes the safe operating region (§ 4.3). Dispatcher IPC serves as the control signal: IPC below ipc_l indicates *Demand*, IPC above ipc_h indicates *Supply*, and values within the interval correspond to *Maintain*. This design ensures that resource allocation is driven by dispatch-path efficiency.

For the CPU computation partition (right FSM in Fig. 12), which may host one or more processes, the FSM determines its state based on both resource utilization (e.g., LLC occupancy) and observed performance (LLC miss rate, and IPC). If the LLC occupancy remains low, the partition enters *Supply* state, indicating underutilized capacity. As occupancy increases, the partition transitions from *Supply* to *Maintain*, but does not immediately trigger *Demand*, since poor locality may limit the benefit of additional cache. Instead, *Demand* is triggered only when performance degradation is observed (e.g., reduced IPC or increased cache misses). If allocating additional LLC fails to improve performance, the partition returns to *Maintain*, ensuring resources are allocated only when they provide measurable benefit.

The controller follows a priority-based policy to resolve competing demands (Algorithm 2). At each decision period Δt , the controller first updates the FSM states based on runtime measurements (line 5). It then determines the change in resource allocation for the dispatcher partition. When only one partition signals *Demand*, resources are reallocated to that partition. When both partitions signal *Demand*, the dispatcher is prioritized to prevent GPU stalls (line 6). To ensure stability, SuperGuard applies a filtering mechanism that requires allocation decisions to persist for N_{stable} consecutive intervals before taking effect (line 9-11). This avoids oscillations caused by transient fluctuations and ensures stable control behavior over time.

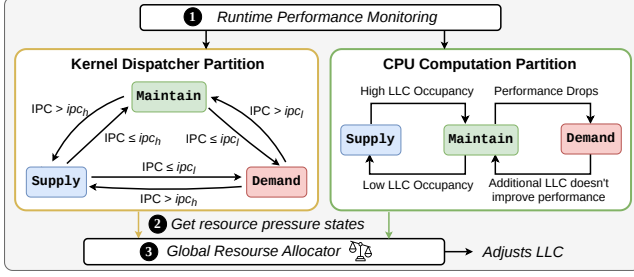


Figure 12. Runtime controller with FSM-based resource pressure modeling for both kernel dispatcher and CPU partition.

Algorithm 2: Priority-based Resource Allocation

```

1 Function RuntimeControl( $w_{init}, W_{max}, \Delta t, N_{stable}$ ):
2    $w \leftarrow w_{init}, \delta_{prev} \leftarrow 0, n \leftarrow 0$ 
3   while true do
4     ApplyCache( $w, W_{max} - w$ ) and Sleep( $\Delta t$ )
5     ( $S_d, S_c$ )  $\leftarrow$  (UpdateState( $disp$ ), UpdateState( $cpu$ ))
6     // Determine adjustment (prioritize dispatcher
7       when both demand)
8      $\delta \leftarrow \begin{cases} +1 & \text{if } S_d = \text{DEMAND} \\ -1 & \text{else if } S_c = \text{DEMAND} \\ 0 & \text{otherwise} \end{cases}$ 
9      $n \leftarrow (\delta = \delta_{prev}) ? n + 1 : 1$ 
10     $\delta_{prev} \leftarrow \delta$ 
11    if  $n \geq N_{stable}$  then
12       $w \leftarrow \text{clamp}(w + \delta, 1, W_{max})$ 
13       $\delta_{prev} \leftarrow \text{None}$ 

```

5 Implementation

SuperGuard is implemented as a lightweight runtime controller in Python that can be readily integrated with common ML frameworks such as vLLM [30] and DeepSpeed [47]. The system consists of approximately 1.5K lines of code. To enforce resource isolation and control, SuperGuard leverages Arm Memory System Resource Partitioning and Monitoring (MPAM) [4]. MPAM requires a kernel patch (we use NV-Kernels 24.04-linux-nvidia-adv-6.11) and is exposed through Linux `resctrl`, where partitions are represented as directories. It provides a unified interface to control CPU core affinity, LLC capacity, and DRAM bandwidth. LLC partitioning uses a bitmask specifying the subset of LLC ways assigned to each partition. SuperGuard dynamically adjusts these bitmasks at runtime to control LLC allocation. LLC occupancy is monitored via MPAM’s cache-storage usage interface, which reports per-partition utilization. Additional signals, including dispatcher IPC and cache miss rate, are collected using Linux `perf`.

Although MPAM also provides memory bandwidth control, we find it ineffective in our setting. It operates as a soft isolation mechanism enforcing average limits over a time window, rather than strict instantaneous constraints, failing to regulate bursty memory requests that are critical to kernel dispatch latency. Enabling it degrades CPU performance with negligible dispatch benefit. MPAM also supports

priority partitioning (SDPRI), which assigns Quality of Service (QoS) levels to partitions. While potentially useful for finer-grained control, it is not currently available on NVIDIA Grace Hopper Superchips and is therefore not evaluated.

6 Evaluation

6.1 Experimental Setup

6.1.1 Hardware. We conduct all experiments on NVIDIA GH200 Superchips, which integrate a Hopper GPU with 144GB of HBM memory and Grace CPU via NVLink-C2C. Each CPU provides 72 cores and a 12-way LLC.

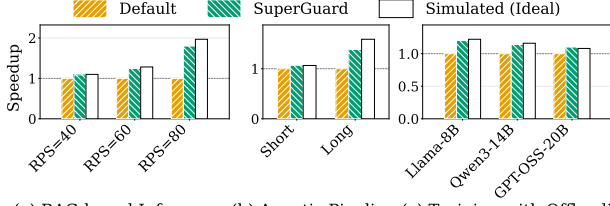
6.1.2 Workloads. We evaluate SuperGuard on three representative workload families involving CPU–GPU co-execution: (1) RAG-based LLM inference, (2) agentic pipelines with external tool execution, and (3) LLM training with CPU offloading. These workloads capture real-world scenarios where CPU-side computation overlaps with GPU execution and can impact end-to-end performance.

RAG-based LLM Inference. We evaluate an asynchronous RAG pipeline built using LangChain [9], a widely adopted open-source framework for LLM applications, together with vLLM (v0.15.1) [30] and Faiss [15]. We use Qwen3-14B [68] as the generation model and construct a vector database from QG-Bench [61]. Each request consists of a question and retrieved context from the vector database. We retrieve the top-3 documents and prepend them to the prompt. Requests arrive according to a Poisson process with a configurable rate, and we vary the retrieval cost using different search parameters ($nprobe$ from 64 to 256). The system uses multi-threaded CPU execution for retrieval and continuous batching on GPU via vLLM. Each experiment runs for 20 seconds at a fixed request rate, and we measure per-request performance.

Agentic Pipeline. We evaluate an agentic pipeline where the LLM interacts with external tools via function calls. Specifically, we construct an Agent-SQL workload where the model generates SQL queries executed on the CPU, and the results are fed back to the vLLM for final response generation. We use Qwen3-30B-A3B [59] as the model and DuckDB [45] as the database engine. Queries are drawn from the BIRD-SQL Mini-Dev [32] dataset and categorized into short and long queries based on execution time (e.g., a 50ms threshold).

LLM Training with CPU Offloading. We evaluate training workloads using DeepSpeed (v0.18.9) with SuperOffload [33] enabled, where optimizer computation is offloaded to the CPU and overlaps with GPU execution. We evaluate both dense (Llama-3.1-8B [60] and Qwen3-14B [6]) and sparse MoE (GPT-OSS-20B [42]) models using a subset of the Alpaca dataset [58] with sequence length 2K.

6.1.3 Hyperparameters. The decision period Δt is set based on workload iteration time: 0.5s for inference (typical request completion time) and 5s for training (training step duration). This ensures the controller can respond within one



(a) RAG-based Inference (b) Agentic Pipeline (c) Training with Offloading

Figure 13. End-to-end performance across inference and training workloads. SuperGuard restores performance lost to CPU-GPU interference, consistently outperforming the Default baseline and approaching the Simulated ideal configuration. Gains are largest under high CPU pressure, where interference most severely disrupts GPU execution.

execution cycle while avoiding excessive sampling overhead. We set ϵ_{sat} to 1%-5% and ϵ_{deg} to 10%-20% based on SLO targets and the observed performance trend curves (Fig. 11). For the CPU partition FSM, we use a 5% threshold for IPC and LLC miss rate changes (a standard sensitivity threshold in performance analysis) and 90% for high LLC occupancy. The system is robust to moderate variations in these thresholds.

6.1.4 Metrics. For inference (RAG and agentic pipelines), we measure latency using end-to-end time-to-first-token (TTFT(full)), time-between-tokens (TBT), and GPU-side first-token generation time (TTFT(gen)) excluding host-side latency (e.g., retrieval). For training, we report TFLOPS throughput. For inference, we further evaluate performance under SLOs by reporting SLO attainment rates. Additional configuration details are in Appendix F.

6.1.5 Baselines. We evaluate SuperGuard on all three workloads (RAG-based inference, agentic pipelines, and training with offloading) against two baselines. *Default*: The default system configuration with no resource isolation or partitioning applied. *Interference-free (Simulated)*: An upper-bound configuration that removes CPU-side contention by replaying pre-computed host-side results via trace simulation, approximating an interference-free execution without concurrent CPU interference.

6.2 End-to-end Effectiveness

We evaluate the end-to-end effectiveness of SuperGuard across RAG-based inference, agentic pipelines, and training with CPU offloading. As shown in Fig. 13, SuperGuard consistently improves performance over the *Default* baseline, achieving up to 1.8× speedup and recovering up to 95% of the interference-free *Simulated* upper bound. Gains are largest under high CPU pressure, where interference most severely disrupts GPU execution.

RAG-based inference. As request rate increases, CPU-side retrieval introduces significant memory contention, which slows kernel dispatch and increases GPU idle gaps. SuperGuard consistently reduces end-to-end latency, with larger gains at higher request rates where interference is strongest.

Agentic pipeline. In the agentic SQL workload, CPU-side tool execution introduces additional memory contention beyond retrieval. SuperGuard improves performance for both short and long queries, with larger gains for long queries that impose higher CPU demand, demonstrating its effectiveness in more complex, tool-augmented workloads.

Training with CPU offloading. In training, CPU optimizer execution introduces sustained memory traffic that interferes with GPU execution. SuperGuard improves throughput across models by stabilizing kernel dispatch and reducing GPU idle time, demonstrating that its benefits extend beyond inference to throughput-oriented training workloads.

Overall, SuperGuard achieves up to 1.8× performance improvement and recovers up to 95% of the interference-free upper bound, with the largest gains observed under high CPU contention. Additional results across workload configurations are shown in Appendix G.

6.3 SLO Attainment

Beyond end-to-end performance, we evaluate the impact of Superchip interference on service-level objectives (SLOs), which are critical in real-world deployments. We focus on inference and measure SLO attainment under increasing load. When the host-side workload becomes more intensive through higher request rates (Fig. 14(a) and (c)) or increased retrieval cost (Fig. 14(b)), CPU-side memory contention intensifies, reducing dispatch efficiency and degrading GPU execution. As a result, the *Default* system’s TBT and TTFT(gen) SLO attainment drops significantly. In contrast, SuperGuard significantly delays when SLO attainment begins to drop, and maintains performance close to the interference-free upper bound. Improvements are also reflected in TTFT(full), which includes retrieval latency. Although TTFT(full) is dominated by host-side computation, SuperGuard still improves end-to-end latency by reducing GPU-side delay.

6.4 Understanding Why SuperGuard Works

6.4.1 How does SuperGuard maintain sufficient dispatch rate? We analyze how SuperGuard maintains a sufficient kernel dispatch rate by measuring LLC miss rate and IPC. Fig. 15 shows that under Superchip interference, concurrent CPU workloads increase LLC miss rate for the dispatcher, making it increasingly memory-bound. As a result, dispatcher IPC drops, reducing the kernel dispatch rate. By isolating the dispatcher from competing CPU workloads, SuperGuard reduces LLC miss rate by 60.6% on average compared to the *Default* baseline, and increases dispatcher IPC by 1.32×. This restores a sufficient kernel dispatch rate and prevents the GPU execution queue from draining.

Importantly, SuperGuard does not aim to fully eliminate contention or match the interference-free *Simulated* setting. Instead, it allocates only the necessary resources required to sustain a sufficient dispatch rate. As a result, the dispatcher’s LLC miss rate remains higher than the *Simulated* baseline

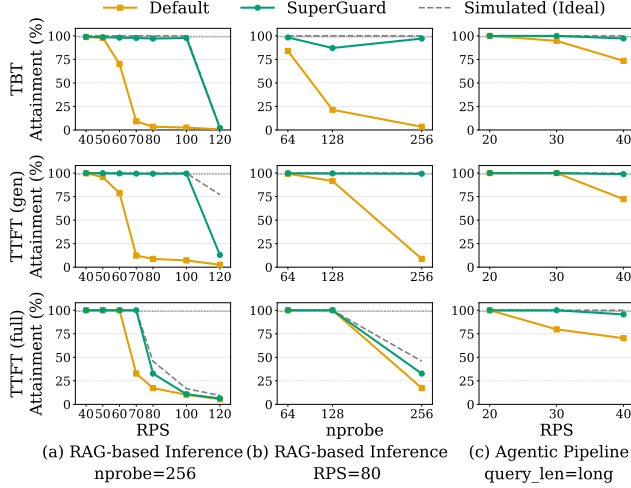


Figure 14. SLO attainment comparison for RAG-based LLM inference and Agent-SQL systems. SuperGuard achieves significant improvements in TBT and TTFT SLO attainment compared to baselines as workload intensity increases.

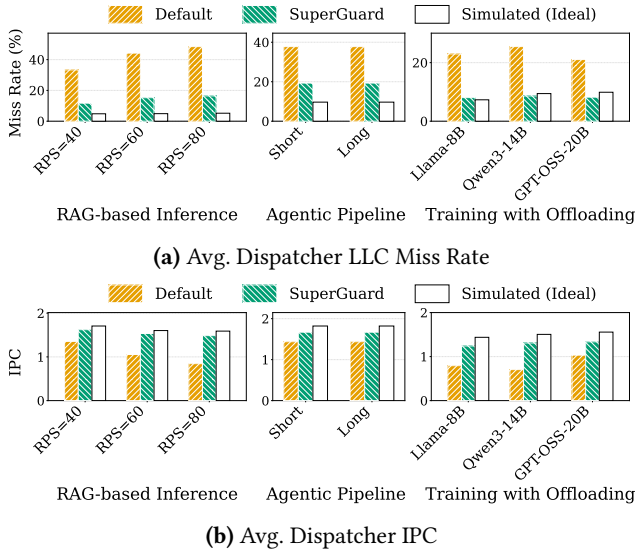


Figure 15. SuperGuard mitigates memory interference and restores dispatcher efficiency. (a) LLC miss rate decreases, indicating reduced cache contention. (b) Dispatcher IPC is restored close to the interference-free upper bound.

(1.8 $\times$ on average), while still achieving near-optimal GPU utilization. This demonstrates that SuperGuard avoids over-provisioning shared CPU resources while effectively protecting dispatch performance.

6.4.2 How does SuperGuard affect concurrent CPU application progress? We next evaluate the impact of SuperGuard on co-located CPU applications. Fig. 16 shows the normalized CPU-side computation latency, including retrieval time for RAG-based inference and SQL query time for

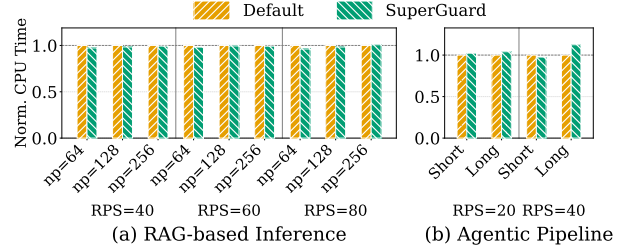


Figure 16. Impact of SuperGuard on CPU-side computation latency across workloads and configurations.

the agentic SQL workload. Across all configurations, SuperGuard introduces negligible overhead to CPU applications, with an average slowdown of only 0.5% and a worst-case overhead of 12.5%. This small overhead arises because SuperGuard allocates only the necessary LLC resources required to sustain a sufficient kernel dispatch rate, rather than fully isolating the dispatcher. As a result, most CPU-side computation continues to execute with near-native performance. Overall, these results demonstrate that SuperGuard effectively mitigates CPU-GPU interference while preserving the progress of concurrent CPU workloads, achieving a balance between GPU efficiency and CPU performance.

6.4.3 How does SuperGuard adjust LLC allocations at runtime when workload changes? Fig. 17 shows SuperGuard’s dynamic response to a sudden increase in request rate in RAG-based inference. It illustrates how SuperGuard adapts LLC allocation at runtime to maintain a sufficient dispatch rate under changing workloads. In this experiment, we set `stable_required=2`. We first run the system at RPS=40 for 120 seconds, allowing SuperGuard to complete profiling and reach a steady control phase. We then increase the request rate to RPS=80 and continue execution.

During steady-state operation at RPS=40, allocating 2–3 LLC ways to the dispatcher is sufficient to keep its IPC within the safe region. Despite minor runtime fluctuations, SuperGuard maintains dispatcher IPC above the lower threshold `ipc_1`, which ensures stable GPU execution. When the request rate increases, the resulting memory pressure causes an immediate drop in dispatcher IPC. In response, SuperGuard detects the deviation and increases LLC allocation to the dispatcher. Although concurrent CPU workloads (e.g., vector search) also demand additional cache resources, SuperGuard prioritizes the dispatcher to preserve kernel launch rate. Within approximately 7 seconds, the system reaches a new steady state, allocating 7–8 LLC ways to the dispatcher. This restores dispatcher IPC to the safe region, stabilizes kernel dispatch, and prevents GPU idle time. These results demonstrate that SuperGuard can rapidly adapt to workload changes, dynamically balancing resource allocation to maintain dispatch efficiency without over-provisioning.

6.4.4 Is dynamic partitioning necessary? We compare SuperGuard with a static partitioning policy that allocates

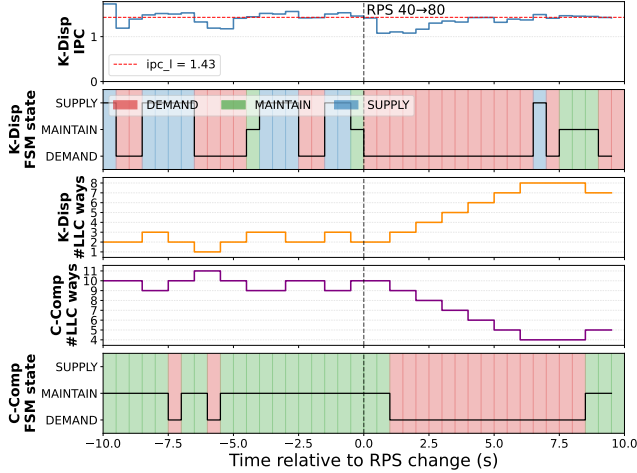


Figure 17. Runtime adaptation of SuperGuard under a workload change (RPS 40 → 80). A spike in request rate reduces dispatcher IPC due to increased LLC contention. SuperGuard detects this drop and increases LLC allocation to the dispatcher, restoring IPC above the safe threshold (ipc_l) and stabilizing execution. The system converges to a new steady state within a few seconds while avoiding over-provisioning. Background colors indicate resource states: red (DEMAND), blue (SUPPLY), green (MAINTAIN).

fixed LLC based on profiling results (2 ways for the dispatcher and 10 ways for CPU workloads). This static configuration is sufficient under the initial workload (RPS=40), but cannot adapt to runtime changes. Fig. 18 shows dispatcher IPC and GPU performance (TBT and TTFT(gen)) during a workload shift from RPS=40 to RPS=80. While static partitioning maintains reasonable performance under low load, it fails to adapt when contention increases, causing a sustained drop in dispatcher IPC and significant GPU performance degradation. In contrast, SuperGuard dynamically adjusts resource allocation, quickly restoring dispatcher IPC and maintaining stable GPU performance after the workload change. This demonstrates that static partitioning is insufficient under dynamic workloads, and adaptive control is necessary to sustain performance under varying interference conditions.

6.4.5 How do workload and model configurations affect SuperGuard? We study sensitivity to workload and model configurations across both inference and training workloads. For inference, we vary retrieval cost ($nprobe$) to control CPU-side memory pressure.

Fig. 19 shows performance speedup comparison under different workload configurations.

Effect of workload intensity. The relative performance gain increases with workload intensity (i.e., higher $nprobe$ values in RAG-based inference and higher request rates in agentic pipelines). This trend is expected because more intensive host-side workloads cause more severe memory contention with the kernel dispatcher, intensifying CPU-GPU interference. For training, we vary batch size and model scale.

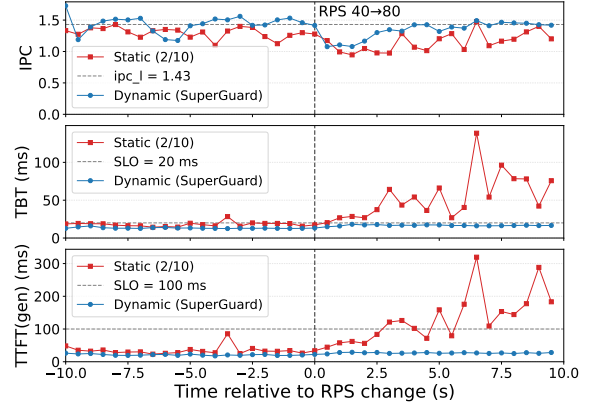


Figure 18. Dynamic vs. static partitioning under workload change (RPS 40 → 80) in RAG-based inference ($nprobe=256$). Under increased load, static partitioning fails to maintain dispatcher IPC, leading to significant GPU performance degradation (TBT and TTFT(gen)) and SLO violations. In contrast, SuperGuard dynamically adjusts resource allocation, restoring dispatcher IPC and maintaining stable performance.

Fig. 20 shows GPU throughput under varying batch sizes and model scales, with sequence length fixed at 2K.

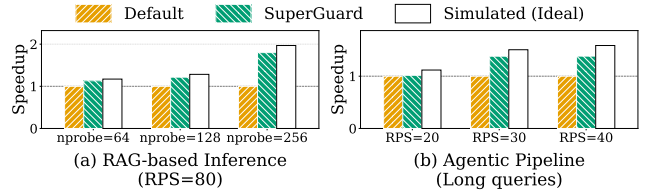


Figure 19. Performance speedup comparison under different workload configurations ($nprobe$ for RAG-based Inference, RPS for Agentic pipeline).

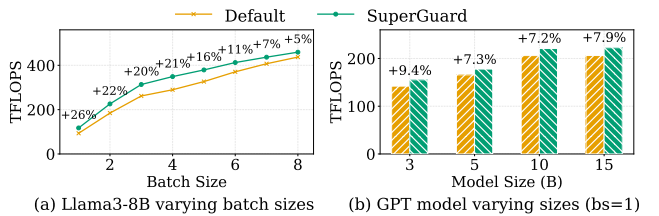


Figure 20. GPU throughput comparison under different model configurations (batch size and model size) in LLM training with CPU offloading.

Effect of batch size. SuperGuard consistently improves throughput across all batch sizes, but the relative performance gain decreases as batch size increases (25.8% at BS=1 to 5.1% at BS=8). This trend arises because larger batch size increase GPU kernel execution time without proportionally increasing the number of kernel launches of dispatcher tasks. As a result, GPU execution becomes less sensitive to dispatch latency, reducing the impact of CPU-GPU interference.

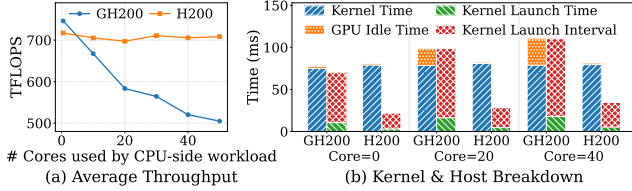


Figure 21. GPU throughput under concurrent CPU execution on GH200 and H200. (a) H200 throughput remains stable under high CPU load. (b) Breakdown of GPU execution showing that H200 host-side launch delay does not lead to growing GPU idle time.

Effect of model size. We evaluate different model sizes by varying the hidden dimension and number of transformer layers (Appendix F.3). SuperGuard maintains consistent performance improvements across all scales, as the number of dispatcher tasks per layer and their memory access patterns remain similar, and the dispatch path remains sensitive to memory contention. As a result, SuperGuard effectively mitigates interference regardless of model size.

Overall, these results show that SuperGuard is robust across model configurations, with benefits most noticeable when GPU execution is more sensitive to dispatch latency.

6.4.6 What is the overhead of SuperGuard’s controller?

SuperGuard introduces negligible overhead (0.1% to 0.4%) across workloads. This is achieved through lightweight monitoring and control using system interfaces (Linux perf and MPAM resctrl) and moderate decision period ($\geq 500\text{ms}$), avoiding frequent or intrusive interventions.

6.4.7 Behavior on loosely coupled systems.

We evaluate the same controlled micro-benchmark (§ 2.1) on a loosely coupled GPU system (H200) to examine whether dispatch-path interference also appears in conventional PCIe-based architectures. As shown in Fig. 21(a), GPU throughput on H200 remains relatively stable compared to the GH200 Superchip. Correspondingly, GPU idle time shows little change as CPU load increases (Fig. 21(b)). Similar to GH200, host-side kernel launch delay increases with CPU load, indicating growing memory contention. However, in this setting, the host-side dispatch process remains sufficiently fast to keep the GPU continuously fed, and no additional idle gaps are introduced. These results indicate that dispatch-path interference exists in conventional systems but is less pronounced. In loosely coupled architectures, the dispatch rate remains above the GPU’s consumption rate, masking the effect. In contrast, tightly-coupled CPU-GPU architectures like Superchip, exposes this dependency and leads to dispatch-path interference caused performance degradation. Finally, as GPU performance continues to outpace CPU dispatch capability, maintaining a sufficient dispatch rate will become increasingly challenging even in conventional systems.

7 Related Work

Superchip Performance Characterization. Recent work characterizes Superchip architectures, focusing on memory management and data movement over high-bandwidth interconnects (e.g., NVLink-C2C), and performance on HPC workloads [18, 53, 64]. Prior studies also examine CPU-GPU interactions. Vellaisamy et al. show that the CPU can become LLM inference bottleneck under certain configurations [62], while Antepara et al. demonstrate that contention through the GPU-CPU unified memory interface can cause performance degradation [3]. However, these studies primarily characterize coarse-grained performance and do not identify how CPU-side resource contention degraded GPU execution.

Cache and Memory Partitioning. Managing shared resources is critical to prevent performance degradation caused by contention. Hardware mechanisms such as Intel Cache Allocation Technology (CAT) [26], Arm Memory System Resource Partitioning Monitoring (MPAM) [4], and AMD QoS Extensions [2] enable partitioning of LLC, driving extensive work on interference mitigation. Prior studies uses LLC partitioning to improve fairness and throughput in multi-tenant environments [16, 20, 52, 54, 65], and to enable workload consolidation in cloud settings [17, 24, 55, 66], where multiple CPU workloads share resources on the same machine. More comprehensive approaches jointly manage LLC, memory bandwidth and other shared resources to meet SLOs [12, 13, 34, 44] or improve fairness [11, 43, 51]. However, these works target CPU-only workloads and optimize CPU-side performance metrics. They do not consider heterogeneous CPU-GPU execution, where CPU-side resource contention can indirectly limit GPU utilization.

8 Conclusion

In this paper, we identify an emerging performance bottleneck in tightly coupled CPU-GPU Superchips: host-side memory contention can stall GPU execution by slowing the kernel dispatch path. To the best of our knowledge, this is the first work to systematically characterize this effect and expose its root cause. We propose SuperGuard, a lightweight runtime that protects the dispatch path through adaptive resource partitioning. By maintaining a sufficient kernel dispatch rate with minimal resource allocation, SuperGuard restores performance close to the interference-free upper bound across both inference and training workloads, and prevents interference-induced SLO degradation under high load. Our results demonstrate that eliminating bandwidth bottlenecks alone is insufficient for achieving high performance on tightly coupled systems. As CPU-GPU integration continues to deepen, efficient accelerator execution increasingly depends on managing host-side resource contention.

References

- [1] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiying Zhang. 2024. INFERCEPT: Efficient Intercept Support for Augmented Large Language Model Inference. In *In Proceedings of the 41st International Conference on Machine Learning (ICML'24)*.
- [2] AMD. 2018. AMD64 Technology Platform Quality of Service Extensions. <https://www.amd.com/en/support/tech-docs/amd64-technology-platform-quality-service-extensions>
- [3] Oscar Antepara, Leonid Oliker, and Samuel Williams. 2025. Roofline Analysis of Tightly-Coupled CPU-GPU Superchips: A Study on MI300A and GH200. In *In Proceedings of the SC'25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC-W'25)*.
- [4] ARM. 2026. Memory System Resource Partitioning and Monitoring. <https://developer.arm.com/documentation/107768/0101/>.
- [5] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2023. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *arXiv preprint arXiv:2310.11511* (2023).
- [6] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. 2023. Qwen Technical Report. *arXiv preprint arXiv:2309.16609* (2023).
- [7] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. 2022. Improving Language Models by Retrieving from Trillions of Tokens. In *In Proceedings of International Conference on Machine Learning (ICML'22)*.
- [8] Shiyi Cao, Shu Liu, Tyler Griggs, Peter Schafhalter, Xiaoxuan Liu, Ying Sheng, Joseph E. Gonzalez, Matei Zaharia, and Ion Stoica. 2025. MoE-Lightning: High-Throughput MoE Inference on Memory-Constrained GPUs. In *In Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'25)*.
- [9] Harrison Chase. 2022. LangChain. <https://github.com/langchain-ai/langchain>.
- [10] Hongtao Chen, Weiyou Xie, Boxin Zhang, Jingqi Tang, Jiahao Wang, Jianwei Dong, Shaoyuan Chen, Ziwei Yuan, Chen Lin, Chengyu Qiu, Yuening Zhu, Qingliang Ou, Jiaqi Liao, Xianglin Chen, Zhiyuan Ai, Yongwei Wu, and Mingxing Zhang. 2025. KTransformers: Unleashing the Full Potential of CPU/GPU Hybrid Inference for MoE Models. In *In Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP'25)*.
- [11] Ruobing Chen, Wangqi Peng, Yusen Li, Xiaoguang Liu, and Gang Wang. 2023. Orchid: An online learning based resource partitioning framework for job colocation with multiple objectives. *IEEE Trans. Comput.* (2023).
- [12] Ruobing Chen, Haosen Shi, Yusen Li, Xiaoguang Liu, and Gang Wang. 2023. OLPART: Online learning based resource partitioning for colocating multiple latency-critical jobs on commodity computers. In *In Proceedings of the Eighteenth European Conference on Computer Systems (EuroSys'23)*.
- [13] Shuang Chen, Christina Delimitrou, and José F. Martínez. 2019. PARTIES: QoS-Aware Resource Partitioning for Multiple Interactive Services. In *In Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'19)*.
- [14] Albert Cho, Anish Saxena, Moinuddin Qureshi, and Alexandros Daglis. 2024. CoAXIAL: A CXL-Centric Memory System for Scalable Servers. In *In Proceedings of International Conference for High Performance Computing, Networking, Storage and Analysis (SC'24)*.
- [15] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. *arXiv preprint arXiv:2401.08281* (2024).
- [16] Nosayba El-Sayed, Anurag Mukkara, Po-An Tsai, Harshad Kasture, Xiaosong Ma, and Daniel Sanchez. 2018. KPart: A Hybrid Cache Partitioning-Sharing Technique for Commodity Multicores. In *In Proceedings of 2018 IEEE International Symposium on High Performance Computer Architecture (HPCA'18)*.
- [17] Liran Funaro, Orna Agmon Ben-Yehuda, and Assaf Schuster. 2016. Ginseng: {Market-Driven} {LLC} Allocation. In *In Proceedings of 2016 USENIX Annual Technical Conference (USENIX ATC'16)*.
- [18] Luigi Fusco, Mikhail Khalilov, Marcin Chrapek, Giridhar Chukkappalli, Thomas Schulthess, and Torsten Hoefler. 2024. Understanding Data Movement in Tightly Coupled Heterogeneous Systems: A Case Study with the Grace Hopper Superchip. *arXiv preprint arXiv:2408.11556* (2024).
- [19] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. 2023. Precise Zero-Shot Dense Retrieval without Relevance Labels. In *In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL'23)*.
- [20] Adrian Garcia-Garcia, Juan Carlos Saez, Fernando Castro, and Manuel Prieto-Matias. 2019. LFOC: A lightweight fairness-oriented cache clustering policy for commodity multicores. In *In Proceedings of the 48th International Conference on Parallel Processing (ICPP'19)*.
- [21] Mark Hildebrand, Jawad Khan, Sanjeev Trika, Jason Lowe-Power, and Venkatesh Akella. 2020. AutoTM: Automatic Tensor Movement in Heterogeneous Memory Systems Using Integer Linear Programming. In *In Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'20)*.
- [22] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. 2022. Training Compute-Optimal Large Language Models. *arXiv preprint arXiv:2203.15556* (2022).
- [23] Chris Holt, Mark Heinrich, Jaswinder P Singh, Edward Rothberg, and John Hennessy. 1995. *The Effects of Latency, Occupancy, and Bandwidth in Distributed Shared Memory Multiprocessors*. Technical Report. Stanford University.
- [24] Xiaokang Hu, Yuchao Cao, Naixuan Guan, Yifan Wu, Xishi Qiu, Shengdong Dai, Ben Luo, Sanchuan Cheng, Fudong Qiu, Yibin Shen, et al. 2026. Cacheman: A Comprehensive Last-Level Cache Management System for Multi-tenant Clouds. In *In Proceedings of the 31st ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP'26)*.
- [25] Zhengding Hu, Vibha Murthy, Zaifeng Pan, Wanlu Li, Xiaoyi Fang, Yufei Ding, and Yuke Wang. 2025. HedraRAG: Co-Optimizing Generation and Retrieval for Heterogeneous RAG Workflows. In *In Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP'25)*.
- [26] Intel. 2023. *Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3B: System Programming Guide, Part 2*. Intel. <https://www.intel.com/content/www/us/en/content-details/868147/intel-64-and-ia-32-architectures-software-developer-s-manual-volume-3b-system-programming-guide-part-2.html>
- [27] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models Through Question Complexity. *arXiv preprint arXiv:2403.14403* (2024).
- [28] Wenqi Jiang, Marco Zeller, Roger Waleffe, Torsten Hoefler, and Gustavo Alonso. 2023. Chameleon: A Heterogeneous and Disaggregated

- Accelerator System for Retrieval-Augmented Language Models. *arXiv preprint arXiv:2310.09949* (2023).
- [29] Xuanlin Jiang, Yang Zhou, Shiyi Cao, Ion Stoica, and Minlan Yu. 2024. Neo: Saving gpu memory crisis with cpu offloading for online llm inference. *arXiv preprint arXiv:2411.01142* (2024).
- [30] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *In Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles (SOSP'23)*.
- [31] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich K ttler, Mike Lewis, Wen-tau Yih, Tim Rockt schel, Sebastian Riedel, and Douwe Kiela. 2021. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *arXiv preprint arXiv:2005.11401* (2021).
- [32] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2023. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. In *In Proceedings of Advances in Neural Information Processing Systems (NeurIPS'23)*.
- [33] Xinyu Lian, Masahiro Tanaka, Olatunji Ruwase, and Minjia Zhang. 2026. SuperOffload: Unleashing the Power of Large-Scale LLM Training on Superchips. In *In Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating System (ASPLOS'26)*.
- [34] David Lo, Liquan Cheng, Rama Govindaraju, Parthasarathy Ranganathan, and Christos Kozyrakis. 2015. Heracles: Improving resource efficiency at scale. In *In Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA'15)*.
- [35] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. 2025. Autellix: An Efficient Serving Engine for LLM Agents as General Programs. *arXiv preprint arXiv:2502.13965* (2025).
- [36] Thomas Merth, Qichen Fu, Mohammad Rastegari, and Mahyar Najibi. 2024. Superposition Prompting: Improving and Accelerating Retrieval-Augmented Generation. *arXiv preprint arXiv:2404.06910* (2024).
- [37] Thomas Moscibroda and Onur Mutlu. 2007. Memory performance attacks: denial of memory service in multi-core systems. In *In Proceedings of the 16th USENIX Security Symposium (USENIX Security'07)*.
- [38] NVIDIA. 2024. NVIDIA GH200 Grace Hopper Superchip. <https://www.nvidia.com/en-us/data-center/grace-hopper-superchip/>.
- [39] NVIDIA. 2025. CUDA Programming Guide. <https://docs.nvidia.com/cuda/cuda-programming-guide/>.
- [40] NVIDIA. 2025. NVIDIA DGX Spark: Personal AI Supercomputer Powered by the GB10 Grace Blackwell Superchip. <https://www.nvidia.com/en-us/products/workstations/dgx-spark/>.
- [41] NVIDIA. 2025. NVIDIA GB300 NVL72. <https://www.nvidia.com/en-us/data-center/gb300-nvl72/>.
- [42] OpenAI. 2025. gpt-oss-120b & gpt-oss-20b Model Card. *arXiv preprint arXiv:2508.10925* (2025).
- [43] Jinsu Park, Seongbeom Park, and Woongki Baek. 2019. CoPart: Coordinated Partitioning of Last-Level Cache and Memory Bandwidth for Fairness-Aware Workload Consolidation on Commodity Servers. In *In Proceedings of the Fourteenth EuroSys Conference 2019 (EuroSys'19)*.
- [44] Tirthak Patel and Devesh Tiwari. 2020. Clite: Efficient and qos-aware co-location of multiple latency-critical jobs for warehouse scale computers. In *In Proceedings of 2020 IEEE International Symposium on High Performance Computer Architecture (HPCA'20)*.
- [45] Mark Raasveldt and Hannes M hleisen. 2019. Duckdb: an embeddable analytical database. In *In Proceedings of the 2019 International Conference on Management of Data (SIGMOD'19)*.
- [46] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning. In *In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC'21)*.
- [47] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *In Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD'20)*.
- [48] Siddhant Ray, Rui Pan, Zhuohan Gu, Kuntai Du, Ganesh Ananthanarayanan, Ravi Netravali, and Junchen Jiang. 2024. RAGServe: Fast Quality-Aware RAG Systems with Configuration Adaptation. *arXiv preprint arXiv:2412.10543* (2024).
- [49] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. ZeRO-Offload: Democratizing Billion-Scale Model Training. In *In Proceedings of 2021 USENIX Annual Technical Conference (USENIX ATC'21)*.
- [50] S. Rixner, W.J. Dally, U.J. Kapasi, P. Mattson, and J.D. Owens. 2000. Memory access scheduling. In *In Proceedings of the 27th International Symposium on Computer Architecture (ISCA'00)*.
- [51] Rohan Basu Roy, Tirthak Patel, and Devesh Tiwari. 2021. Satori: Efficient and fair resource partitioning by sacrificing short-term benefits for long-term gains. In *In Proceedings of 2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA'21)*.
- [52] Juan Carlos Saez, Fernando Castro, Graziano Fanizzi, and Manuel Prieto-Matias. 2024. LFOC+: A fair OS-level cache-clustering policy for commodity multicore systems. *arXiv preprint arXiv:2402.07693* (2024).
- [53] Gabin Schieffer, Jacob Wahlgren, Jie Ren, Jennifer Faj, and Ivy Peng. 2024. Harnessing integrated cpu-gpu system memory for hpc: a first look into grace hopper. In *In Proceedings of the 53rd International Conference on Parallel Processing (ICPP'24)*.
- [54] Vicent Selfa, Julio Sahuquillo, Lieven Eeckhout, Salvador Petit, and Mar a E G mez. 2017. Application Clustering Policies to Address System Fairness with Intel's Cache Allocation Technology. In *In Proceedings of 2017 26th international conference on parallel architectures and compilation techniques (PACT'17)*.
- [55] Mohammad Shahradd, Sameh Elnikety, and Ricardo Bianchini. 2021. Provisioning differentiated last-level cache allocations to VMs in public clouds. In *In Proceedings of the ACM Symposium on Cloud Computing (SoCC'21)*.
- [56] Zhihong Shao, Yeyun Gong, Yelong Shen, Minlie Huang, Nan Duan, and Weizhu Chen. 2023. Enhancing Retrieval-Augmented Large Language Models with Iterative Retrieval-Generation Synergy. *arXiv preprint arXiv:2305.15294* (2023).
- [57] Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, Liming Zhu, and Wenjie Zhang. 2025. HydraRAG: Structured Cross-Source Enhanced Large Language Model Reasoning. In *In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP'25)*.
- [58] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An Instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca.
- [59] Qwen Team. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).
- [60] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timoth e Lacroix, Baptiste Rozi re, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [61] Asahi Ushio, Fernando Alva-Manchego, and Jose Camacho-Collados. 2023. Generative Language Models for Paragraph-Level Question Generation. *arXiv preprint arXiv:2210.03992* (2023).

- [62] Prabhu Vellaisamy, Thomas Labonte, Sourav Chakraborty, Matt Turner, Samantika Sury, and John Paul Shen. 2025. Characterizing and Optimizing LLM Inference Workloads on CPU-GPU Coupled Architectures. *arXiv preprint arXiv:2504.11750* (2025).
- [63] vLLM Team. 2025. Architecture Overview. vLLM Documentation, version 0.16.0. https://docs.vllm.ai/en/v0.16.0/design/arch_overview/
- [64] Felix Werner, Marcel Weisgut, and Tilmann Rabl. 2025. Towards memory disaggregation via NVLink C2C: Benchmarking CPU-requested GPU memory access. In *In Proceedings of the 4th Workshop on Heterogeneous Composable and Disaggregated Systems (HCDS'25)*.
- [65] Yaocheng Xiang, Xiaolin Wang, Zihui Huang, Zeyu Wang, Yingwei Luo, and Zhenlin Wang. 2018. DCAPS: Dynamic cache allocation with partial sharing. In *In Proceedings of the Thirteenth EuroSys Conference (EuroSys'18)*.
- [66] Cong Xu, Karthick Rajamani, Alexandre Ferreira, Wesley Felter, Juan Rubio, and Yang Li. 2018. dcat: Dynamic cache management for efficient, performance-sensitive infrastructure-as-a-service. In *In Proceedings of the Thirteenth EuroSys Conference (EuroSys'18)*.
- [67] Yi Xu, Ziming Mao, Xiangxi Mo, Shu Liu, and Ion Stoica. 2024. Pie: Pooling cpu memory for llm inference. *arXiv preprint arXiv:2411.09317* (2024).
- [68] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).
- [69] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jianxin Yang, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Xuejing Liu, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, Zhifang Guo, and Zhihao Fan. 2024. Qwen2 Technical Report. *arXiv preprint arXiv:2407.10671* (2024).
- [70] Weiping Yu, Ningyi Liao, Siqiang Luo, and Junfeng Liu. 2025. RAGDoll: Efficient Offloading-Based Online RAG System on a Single GPU. *arXiv preprint arXiv:2504.15302* (2025).
- [71] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: efficient execution of structured language model programs. In *In Proceedings of the 38th International Conference on Neural Information Processing Systems (NeurIPS'24)*.

A CPU-GPU Overlap Patterns in LLM Workloads

CPU-GPU overlap is common in LLM workloads, and we illustrate three representative examples in Fig. 22. In RAG-based LLM inference, the CPU performs information retrieval while the GPU generates responses. In the agentic pipeline, the GPU generates function calls, the CPU executes them, and the results are returned to the GPU for response generation. When serving multiple concurrent requests, both scenarios naturally exhibit CPU-GPU overlap. In LLM training with CPU offloading, the GPU performs forward and backward propagation along with partial optimizer steps, while the remaining optimizer computation is executed on the CPU, leading to CPU-GPU overlap.

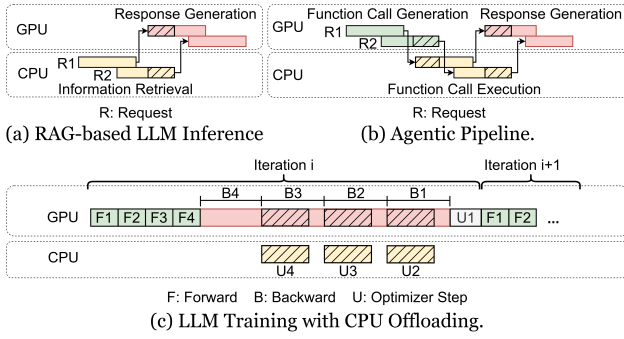


Figure 22. Three execution scenarios involving CPU-GPU interaction in different LLM applications. In (a)-(b), GPU and CPU computations are overlapped when processing multiple concurrent requests. In (c), GPU and CPU computations are overlapped during the backward phase. The numbers indicate the layers.

B Superchip Interference on RAG-based LLM Inference

We present detailed GPU performance results for RAG-based LLM inference experiments under varying request loads and $nprobe$ values. Fig. 23(a) illustrates the workflow. Requests first enter a pre-retrieval CPU-side queue. If CPU cores are already saturated with retrieval computations, requests wait in this queue. After CPU-side retrieval completes, requests proceed to GPU-based generation. Fig. 23(b) illustrates how we breakdown the end-to-end latency. We define TTFT(gen) as the GPU-side first token generation time and define TTFT(full) to cover the host-side latency. We observe that GPU generation performance degrades as request load increases across different $nprobe$ in Fig. 24 and Fig. 25.

C DRAM Bandwidth and Latency in Real Workloads

High DRAM bandwidth usage can inflate the DRAM access latency in real workloads. Fig. 26 shows DRAM behavior

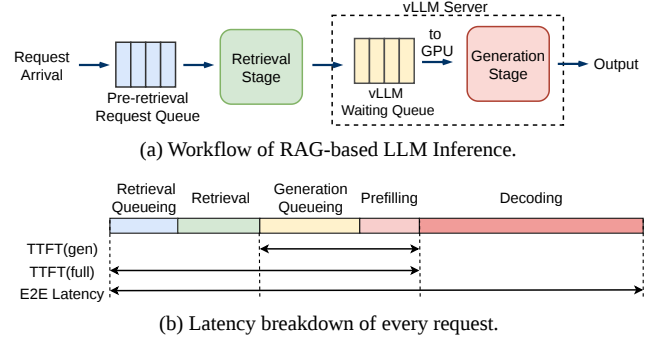


Figure 23. Illustration of RAG-based LLM inference and request latency breakdown.

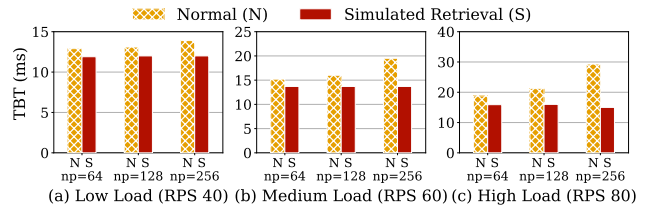


Figure 24. Time between Token (TBT) of the RAG-based inference under different request rates and $nprobe$ (np) values.

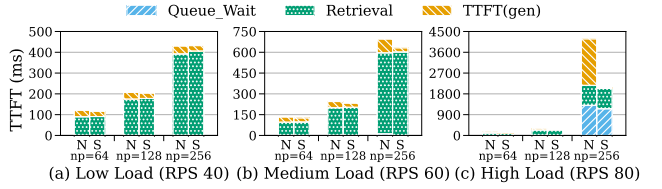


Figure 25. Time to First Token (TTFT) breakdown of the RAG-based inference under different request rates and $nprobe$ (np) values.

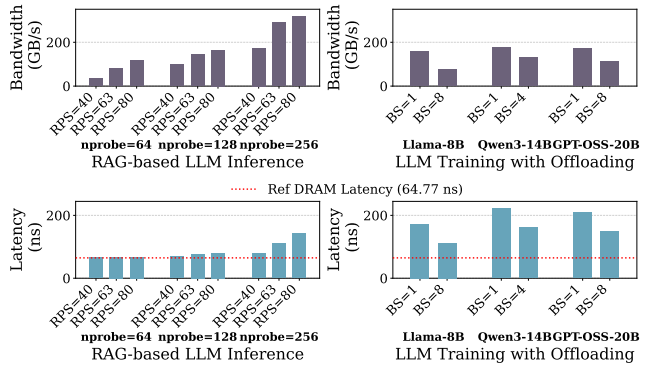


Figure 26. Average DRAM bandwidth usage and access latency for (1) RAG-based inference under different request rates and $nprobe$ values and (2) training with offloading under different models and batch sizes. The red dotted line represents reference DRAM latency at low bandwidth utilization.

under RAG-based LLM inference and LLM training with

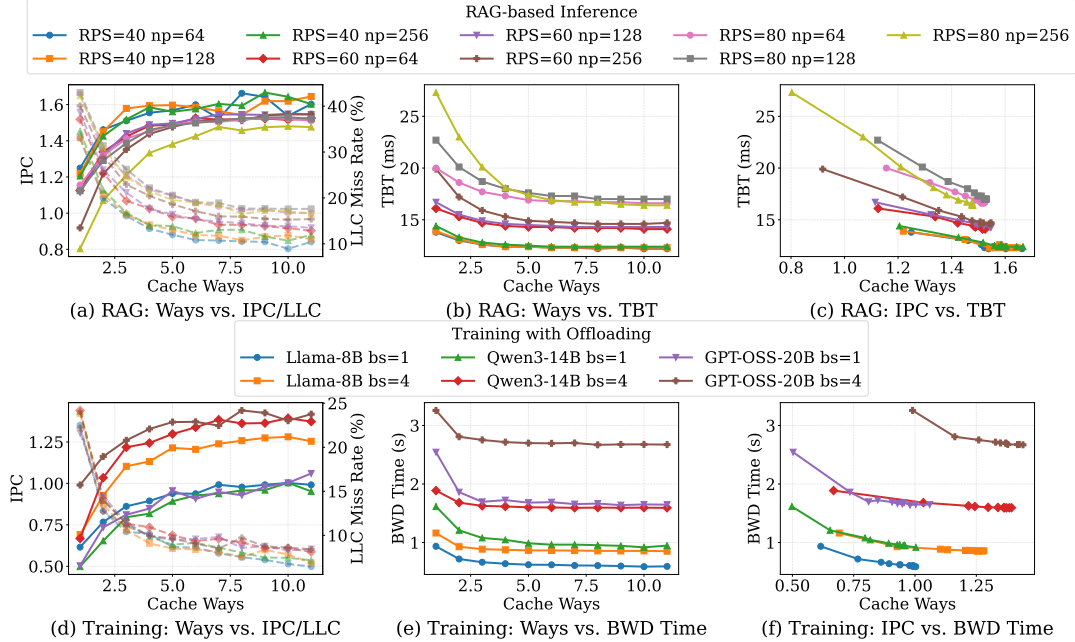


Figure 27. The kernel dispatcher process’s IPC, LLC miss rate, and the GPU performance when different number of LLC ways are allocated for the dispatcher.

CPU offloading. Memory-intensive computations, such as the information retrieval and CPU-side optimizer execution, operate at high DRAM bandwidth (e.g., 77-175GB/s), leading to 1.7× to 3.4× increases in DRAM access latency.

D Threshold Effect

We provide detailed measurements on how the kernel dispatcher IPC, LLC miss rate, and the GPU performance change when we allocate different number of LLC cache ways to the kernel dispatcher partition in Fig. 27. Panels (a) and (b) show that reducing the number of cache ways gradually increases LLC misses and decreases kernel dispatcher IPC. Panels (c) and (d) show that GPU performance drops significantly when allocated cache ways fall below a certain threshold. Panels (e) and (f) plot the relationship between GPU performance and the kernel dispatcher IPC, indicating that the GPU performance degrades sharply when the dispatcher IPC falls below a certain threshold, as the dispatcher fails to maintain a sufficient kernel dispatch rate and causes GPU idling.

E CPU Utilization of Kernel Dispatching

Although many LLM applications require heavy CPU-side computation, the GPU kernel dispatching engine often requires only a small portion of CPU resources. In both RAG-based inference and LLM training with CPU offloading, the ML framework’s CPU utilization is dominated by the GPU worker process, which typically exhibits single-threaded behavior at around 100% utilization on one core per GPU.

vLLM V1 uses a multi-process architecture [63] to minimize host-side overhead and maximize throughput. The

remaining processes of vLLM (e.g., the APIServer process) handle tasks including request scheduling, KV cache managing, and tokenization. However, these tasks only consume minimal CPU resources. In our experiments, vLLM processes consume a peak CPU utilization of only 136% for RAG-based LLM inference and 101% for LLM training with CPU offloading (where 100% represents one fully utilized core), as summarized in Table 1. These measurements confirm that during bootstrap profiling, provisioning a small number of cores to the kernel dispatcher is sufficient.

Table 1. Peak CPU usage of the kernel dispatcher when enough number of cores is allocated to the ML framework. (100% represents one fully utilized core)

	Inference (RAG-based)	Training
Peak CPU Usage	136%	101%

F Detailed Experimental Setup

F.1 RAG-based LLM Inference

Model. We use Qwen3-14B[68] as the primary text generation model in our experiments.

Corpus and Index. We construct a vector database using documents from the context field of QG-Bench[61], a comprehensive benchmark that unifies multiple question answering datasets. We use the first 1M entries from the dataset (1.4M total). For each document chunk, we generate 3584-dimensional semantic vectors using the Qwen2-7B-Instruct-embed-base[69] model. We use the IVF1024 index type, commonly employed for large-scale vector search. We test the

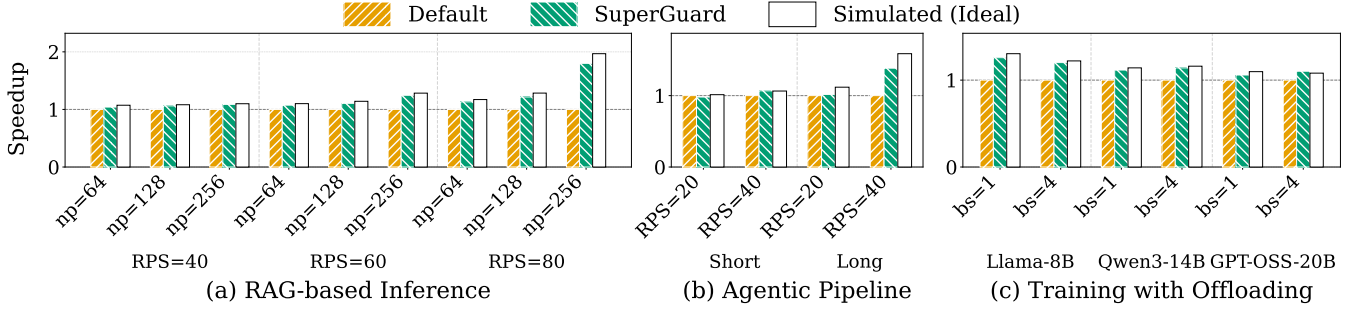


Figure 28. End-to-end performance across inference and training workloads.

RAG pipeline with $nprobe$ values ranging from 64 to 256, representing different search costs and accuracies. This range represents relatively heavy searches.

Workloads. Each request comprises a question from QG-Bench. We retrieve the top 3 most relevant document chunks from the vector database and prepend them to the question to form the LLM prompt. Request arrivals follow a Poisson process with a configurable rate. The end-to-end TTFT(full) and TTFT(gen) SLO is set to 2s and 100ms for responsiveness. The TBT SLO is set to 20 ms for fast generation.

Implementation. We build our RAG pipeline on the state-of-the-art open-source RAG framework LangChain [9], which integrates with vLLM [30] (v0.15.1) for LLM inference and Faiss [15] for vector search. We implement a request-level asynchronous pipeline for the one-shot RAG workflow. To maximize retrieval throughput, we use multi-threading and request batching across all available CPU cores. To maximize GPU utilization, we use multiple vLLM clients to take advantage of vLLM’s continuous batching. For each experiment, we fix a specific request arrival rate, run the RAG system for 20 seconds, and measure per-request performance.

F.2 Agentic Pipeline

Model and Dataset. We use Qwen3-30B-A3B [59] model and the *codebase comments* database from BIRD-SQL Mini-Dev [32] as our dataset. Queries are filtered to remove invalid or extreme outliers, and are categorized by the query duration into short and long queries based on execution latency (e.g., using a 50ms threshold).

Workload and Implementation. We build an Agent-SQL pipeline by using DuckDB [45] as the database engine, which supports high-performance analytical queries via multi-threading. In each step, the LLM issues a SQL query, the CPU executes it via DuckDB, and the result is fed back to the LLM for final response generation. The TTFT(full), TTFT(gen), and TBT SLO are set to 30s, 3s, and 25ms, respectively. The value is longer than the RAG pipeline due to the additional function call generation phase.

F.3 LLM training with CPU offloading

We use SuperOffload [33] (v0.18.9) for heterogeneous training, where the optimizer step on Grace CPU overlaps with

the backward propagation on Hopper GPU. The training workload mainly includes 2 dense models, Llama-3.1-8B [60] and Qwen1-14B [6], and 1 sparse MoE model, GPT-OSS-20B [42]. To understand how the model size affects SuperGuard, we vary the number of hidden dimensions and layers as shown in Table 2. We use a subset of Alpaca dataset [58] for training, with sequence length set to 2K.

Table 2. GPT Model Configurations

# params	# layer	Hidden Dimension
3 billion	60	2048
5 billion	44	3072
10 billion	50	4096
15 billion	78	4096

G Full End-to-End Speedup Results

Fig. 28 shows that SuperGuard consistently improves performance over the *Default* baseline and closely approaches the *Simulated* upper bound across all configurations. The gains are more pronounced under high CPU pressure (e.g., high RPS and large $nprobe$ values).

H Controlled Micro-benchmark Results in Conventional Systems

Fig. 29 shows the controlled micro-benchmark results on the conventional loosely coupled GPU systems, A100 and H200. The host-side dispatch process remains sufficiently fast, indicating the dispatch-path interference is less pronounced.

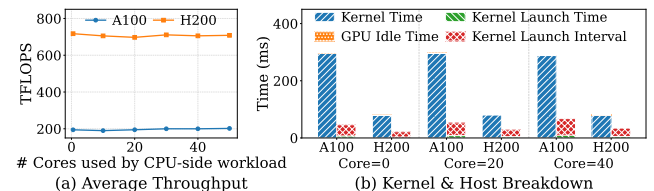


Figure 29. GPU throughput under concurrent CPU execution on A100 and H200.